

This course has already ended.

« Chapter 18.1: Types and implicit values (<https://pl...>) chapter 18.3: Modifying expressions » (<https://plus....>)
CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 18 (https://plus.cs.aalto.fi/studio_2/k2022/w18/)
/ Chapter 18.2: Types in classes, Mixin and Self Type

Chapter 18.2: Types in classes, Mixin and Self Type

About this page:

Key questions How to parameterize types?

Topics Type parameters in classes and traits, Mixin, Self type

What Will I Do? First, read a little and then edit the solution of an earlier task.

Indicative difficulty level: Rather easy

Rough Estimate of Workload:? 2 hours.

Point Value: S100

Related *Projects:* *Interval* 
(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Interval/).

Parametrizing classes and traits

Parameterized classes have been used on the autumn course since the first week when the class `Buffer[T]` was used. The parameterization of classes and traits works much in the same way as the parameterization of functions described in the previous chapter. Below there is an example of an imaginary `UserGroup` class that can handle the same type of users as a group. What kind of users are included in each group is given as a type parameter.

The type parameter is immediately displayed after the trait name and follows the same syntax as what we learned in the previous chapter. A type parameter is available for the entire class. It is an absolute requirement for class methods and variables to work together. In addition to this, it makes method signatures easier and shorter to write than what we saw in the previous chapter, because the type parameter does not need to be re-introduced.

```
trait User {
  def suspendAccount() = ??? // no real implementation here - this is an example
}

class Admin extends User
class Newbie extends User

class UserGroup[T <: User] {
  import scala.collection.mutable.Buffer

  private val userList = Buffer[T]()

  def addUser(user: T) = {
    userList += user
  }

  def suspendAccounts() = {
    userList.foreach(x => x.suspendAccount())
  }

  def lastAccountCreated: T = userList.last
}
```

Exercise: Parameterized Interval, 110p

The purpose of this exercise is to practice the conversion of an existing class into a generic one. This is a relatively normal refactoring operation when you notice that the old code has more use than you first thought.

In the original assignment in chapter 5.1, an Interval (optional) task was performed, which dealt with time interval between two Moment objects. Several methods, such as union, intersection, etc., which are common to any intervals, were implemented in the assignment.

Thus, in this task, the old Interval class (or its model solution  (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_static/interval_model.scala)) is reconsidered and you modify it to be the parameterized version, GenericInterval[T]. Most of the Moment-class functionality that Interval needed to function was related to the comparison of Moment objects. We also generalize this by providing an Ordering[T] object that allows the interval to make exactly the same comparisons.

The task base for the parameterized version is found at the beginning of the chapter, but you should reuse your old code as much as possible.

The property that Ordering[T] cannot offer and which was used by the original Interval is the distance between two "moments". We deliberately leave this out of the class. The length was only used in the class as part of the toString method, so it does not affect other methods.

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 2 (GenericInterval)

Exercise 2 (GenericInterval) has been archived (Thursday, 1 June 2023, 12:00).

Select your files for grading
Show anyway

 **GenericInterval.scala**

Choose File No file chosen

Submit

Mixin

In the previous task, `Ordering[T]` was introduced implicitly, and it was not associated with class `T`. Another option in which classes can be easily associated with a new kind of functionality is called Mixin. It is trait with which one can easily bring a single functionality to a class such that the mixin will implement.

Below is a simple example of mixin that adds *Comparability* to a class. The class must implement only the method `compare`, which returns a positive integer if the parameter is smaller than the element whose method was called, zero if they are the same, and otherwise a negative number.

```
trait Comparable[T] {  
  
  def compare(another: T): Int  
  
  def greater(another: T) = {  
    this.compare(another) > 0  
  }  
  
  def smaller(another: T) = {  
    this.compare(another) < 0  
  }  
}
```

Linking the feature to a class is quite straightforward:

```
class Example(val i: Int) extends Comparable[Example] {  
  
  // this is the only method that needs to be implemented  
  def compare(another: Example) = this.i - another.i  
  
  // ... there is of course much other implementation in the class ...  
}
```

Self type

What if we'd like to add the following method to our mixin `giveLarger`, which returns the larger of two compared items?

```
def giveLarger(another: T): T = {  
  if (this.greater(another))  
    this  
  else  
    another  
}
```

When the method is added to the trait, the compiler reports on an error concerning the return value of the method. The problem is that the type of `this` is `Comparable[T]` and the type of `another` is `T`, so the type of return value is inferred to be `Any`.

Self type is a tag that allows a trait to tell you what type of object it will be a part of. This enables many interesting possibilities for traits:

- The trait can, if necessary, return `this` which is of correct type.
- The trait can call the methods of the class to which it is to be mixed.
- The trait can limit what classes it can be mixed with.

The Self type definition resembles a variable definition ending with `=>`

The specified self-variable is used as `this` variable.

```
trait Comparable[T] {  
  self: T with Comparable[T] =>  
  
  def compare(another: T): Int  
  
  def greater(another: T) = {  
    this.compare(another) > 0  
  }  
  
  def smaller(another: T) = {  
    this.compare(another) < 0  
  }  
  
  def giveLarger(another: T) = {  
    if (this.greater(another))  
      self  
    else  
      another  
  }  
}
```

After this little change `giveLarger` returns the correct type `T` object.

Let us have a quick glimpse of the previously mentioned possibility that the mixin can call the methods of the class (or other mixins) to which it is being mixed. In the example below, the class `Example` implements the method `giveDescription` defined in the trait `Describable`. By specifying the trait `Describable` as a part of `Mixin Printable`'s self type, you can call the method `giveDescription` from `Printable`.

```

trait Describable {
  def giveDescription: String
}

trait Printable[T] {
  self: T with Describable with Printable[T] =>

  def printThis() = {
    println(self.giveDescription)
  }
}

class Example(name: String) extends Describable with Printable[Example] {
  def giveDescription = "Hi! I am " + name
}

```

Summary

- We familiarized with the type parameters in classes and traits.
- We presented the idea of the Mixin design pattern, implementing the features in traits that can be attached to the classes as needed.
- We introducing the self type definition with which a trait can affect the type, of which it will be a part of, when it is linked with the class.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Time spent: (*) Pakollinen Please estimate the total number of minutes you spent on this chapter (reading, assignments, etc.). You don't have to be exact, but if you can produce an estimate to within 15 minutes or half an hour, that would be great.

48

Show anyway

"I feel that I have understood the most important things in this chapter." (*) Pakollinen

- ☒ fully agree
- ☐ somewhat agree
- ☐ somewhat disagree
- ☐ fully disagree
- ☐ I'm unable to answer or don't want to comment.

Submit an update

« Chapter 18.1: Types and implicit values (<https://pl...>

[chapter 18.3: Modifying expressions](https://plus....) » (<https://plus....>